



# Инфраструктура вычислений в биоинформатике

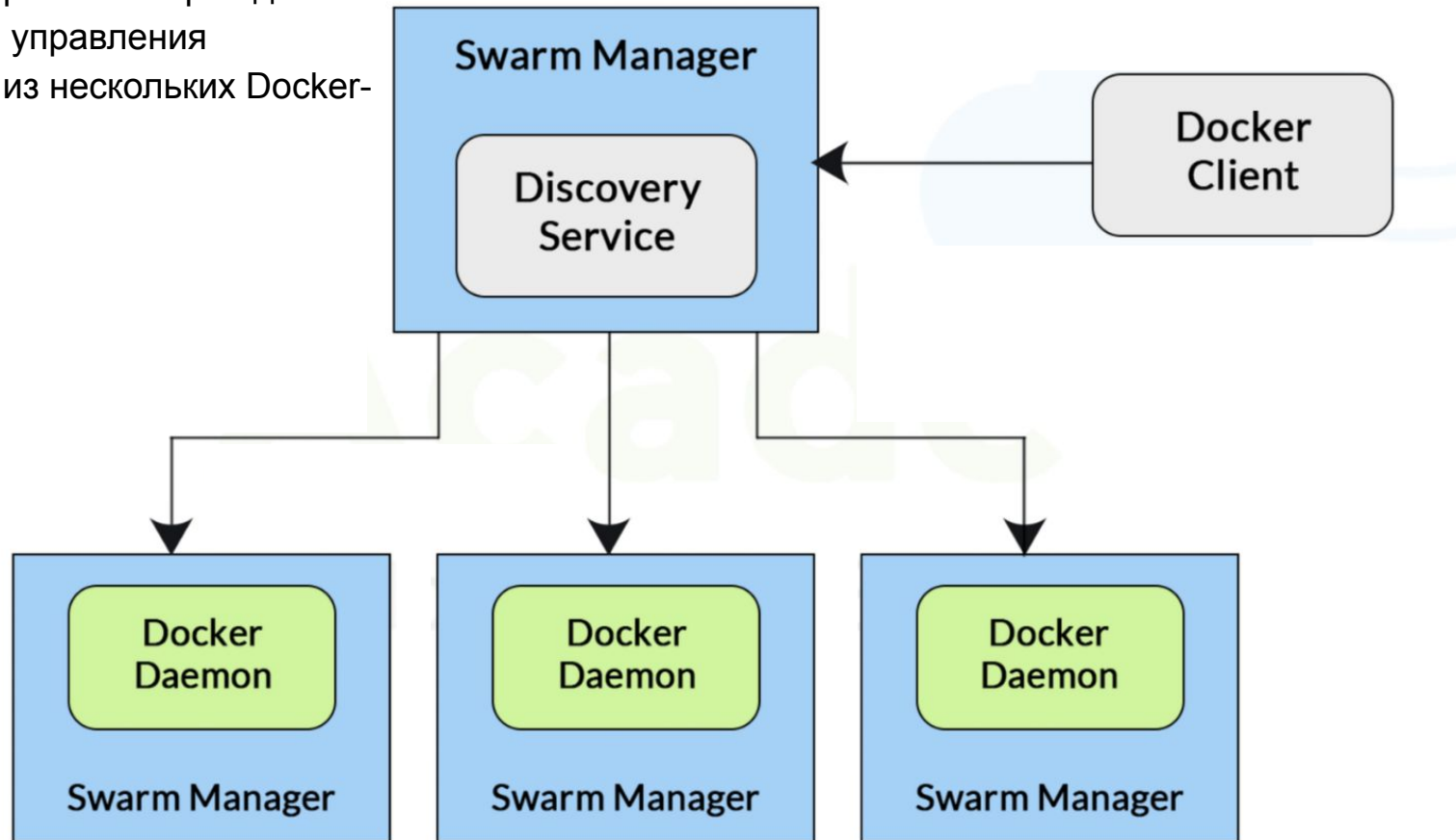
Лекция 7. Продвинутая контейнеризация.

K8s. Экспонирование контейнера в сеть.

Arptainer. Реестры хранения образов.

# Docker Swarm

– встроенный в Docker оркестратор контейнеров для создания и управления кластером из нескольких Docker-хостов.



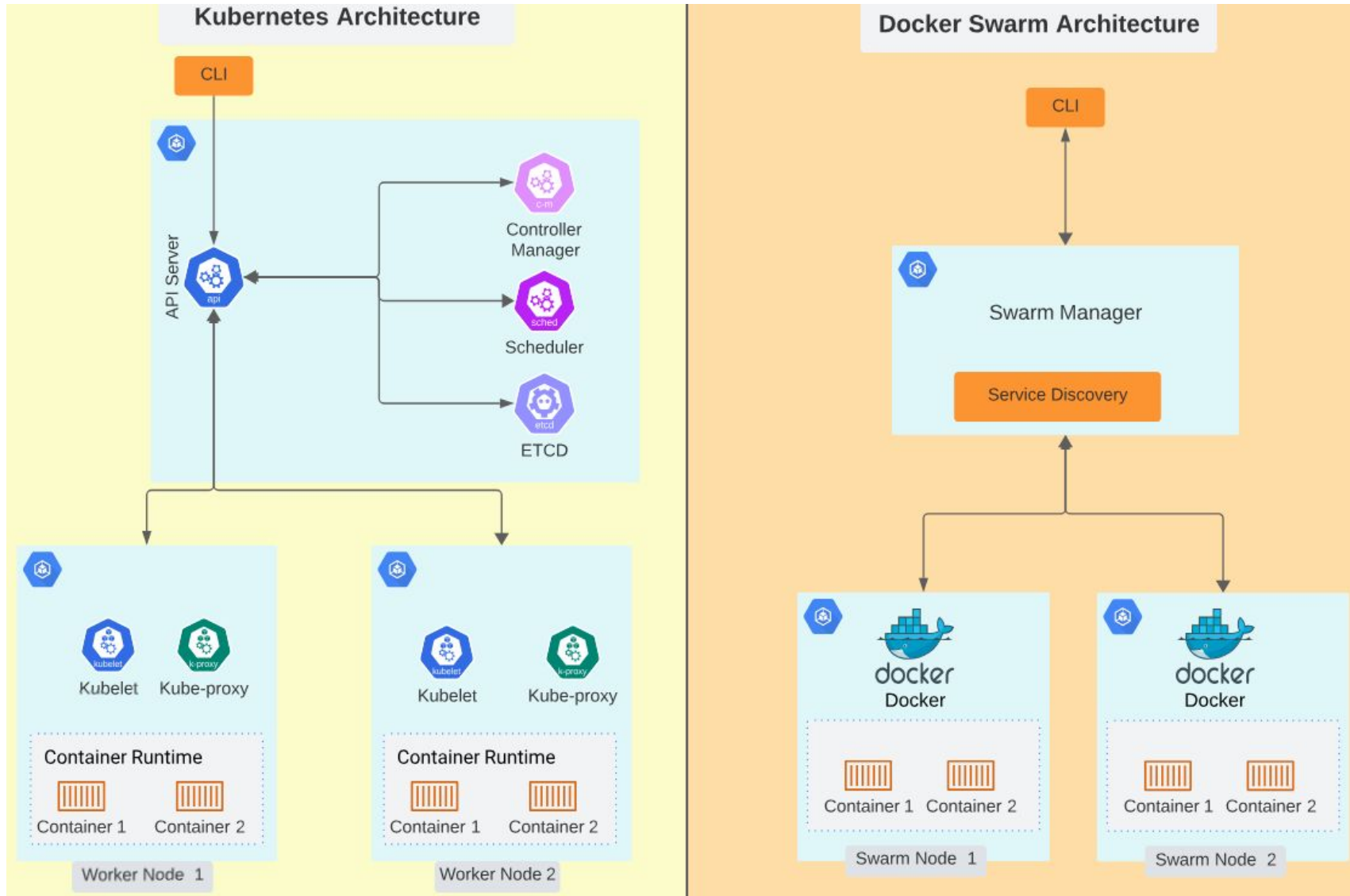
# Зачем нужна оркестрация?

Когда есть микросервисное приложение из 15–20 компонентов. Для каждого нужно:

- выделять ресурсы (CPU, память)
- перезапускать при сбое
- обеспечивать сетевую связность между приложениями
- автоматически масштабировать под нагрузкой
- следить за логами и метриками

Вручную это делать тяжело.

# Kubernetes: почему просто не использовать Docker swarm?



# Базовые понятия: демоны и рантаймы

# Демоны

Демон – это компьютерная программа на хост-системе, которая выполняется как фоновый процесс, не находясь под непосредственным управлением пользователя



# Инструменты запуска контейнеров (runtime)

**runc** – инструмент командной строки для создания и запуска контейнеров в соответствии со спецификацией OCI, утилита использующая библиотеку для взаимодействия с функциями ядра Linux под названием **libcontainer**. **runc** подготавливает среду выполнения для контейнера (создает namespaces и cgroups). Также существует аналогичный инструмент **crun** написанный на Rust, чтобы решить проблемы производительности **runc**. С ним некоторые контейнеры запускаются на 30-60% быстрее.

**containerd** – высокоуровневая среда выполнения контейнеров, демон, использующий runC для запуска контейнеров, реализует также другие высокоуровневые функции, такие как управление образами.



# Демон containerd

— это стандарт среды выполнения контейнеров (container runtime), который управляет полным жизненным циклом контейнеров.

containerd используется платформами контейнеризации и оркестрации приложений в качестве среды выполнения контейнеров

Изначально был извлечен из Docker, теперь containerd является независимым проектом и используется в по умолчанию в платформе **Kubernetes**.

# Kubernetes — оркестратор контейнеров

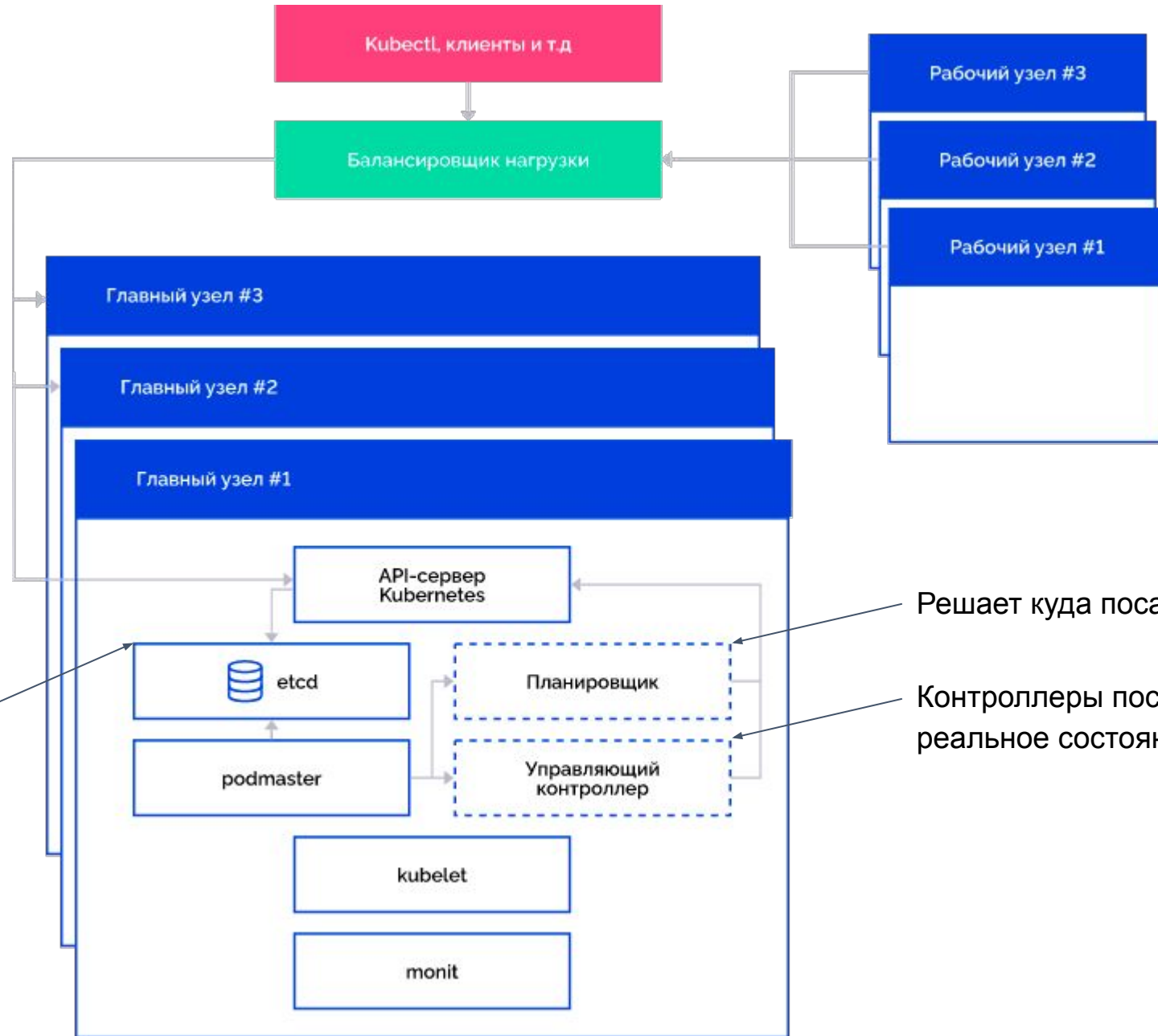
- Под — самая маленькая единица в K8s. Обычно один контейнер = один Pod, но иногда в Pod кладут 2–3 тесно связанных контейнера.



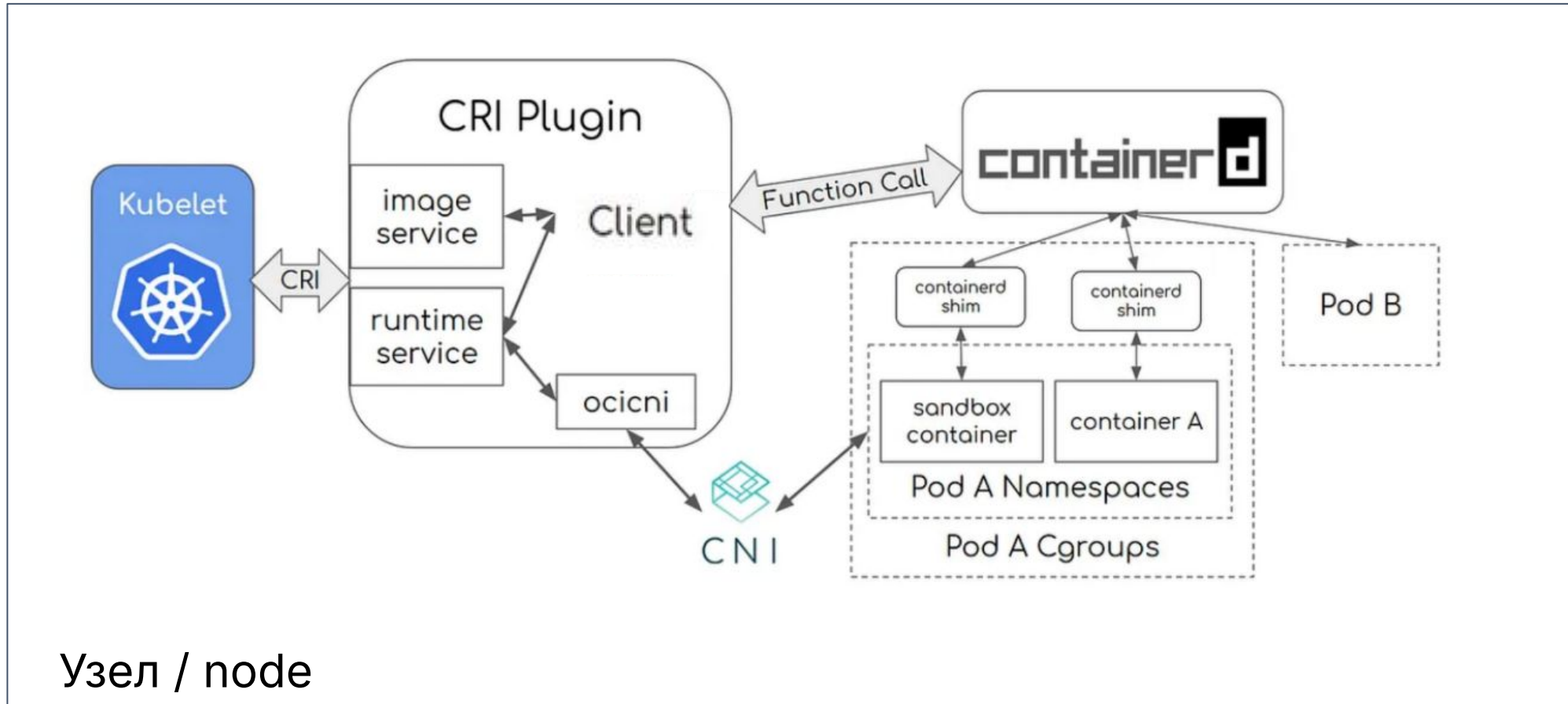
# Архитектура Kubernetes

В сам Kubernetes загружается схема о том, как и что должно работать. K8s отслеживает функционирование кластера и сопоставляет его с желаемым, а если возникает потребность, то заново запускает необходимые компоненты.

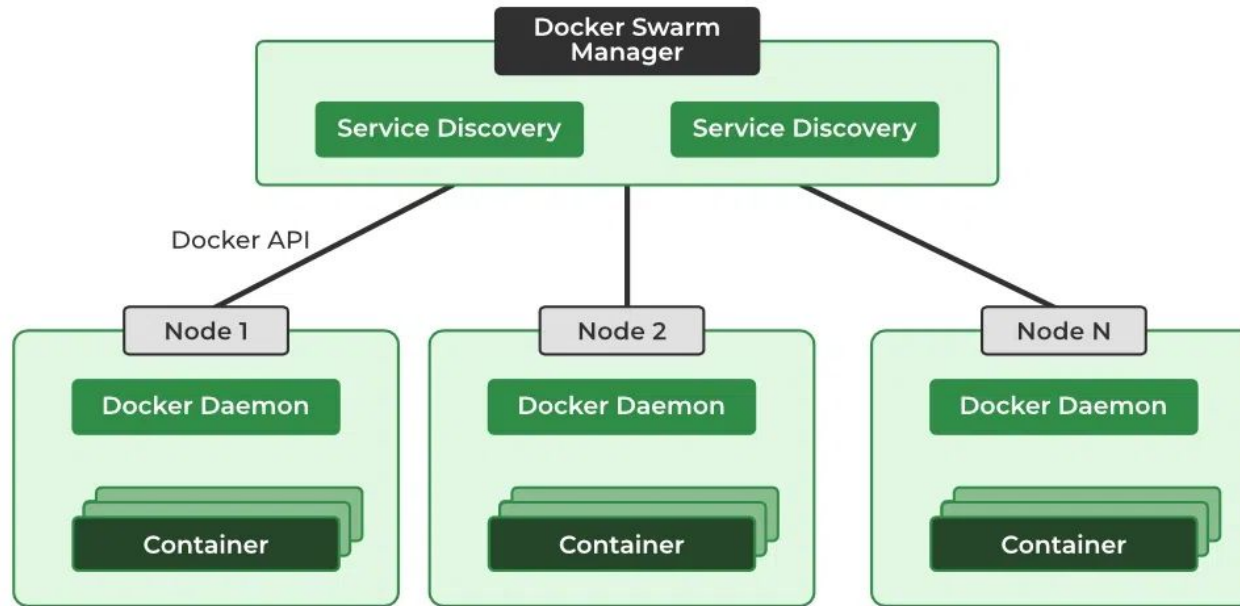
хранит всё состояние кластера



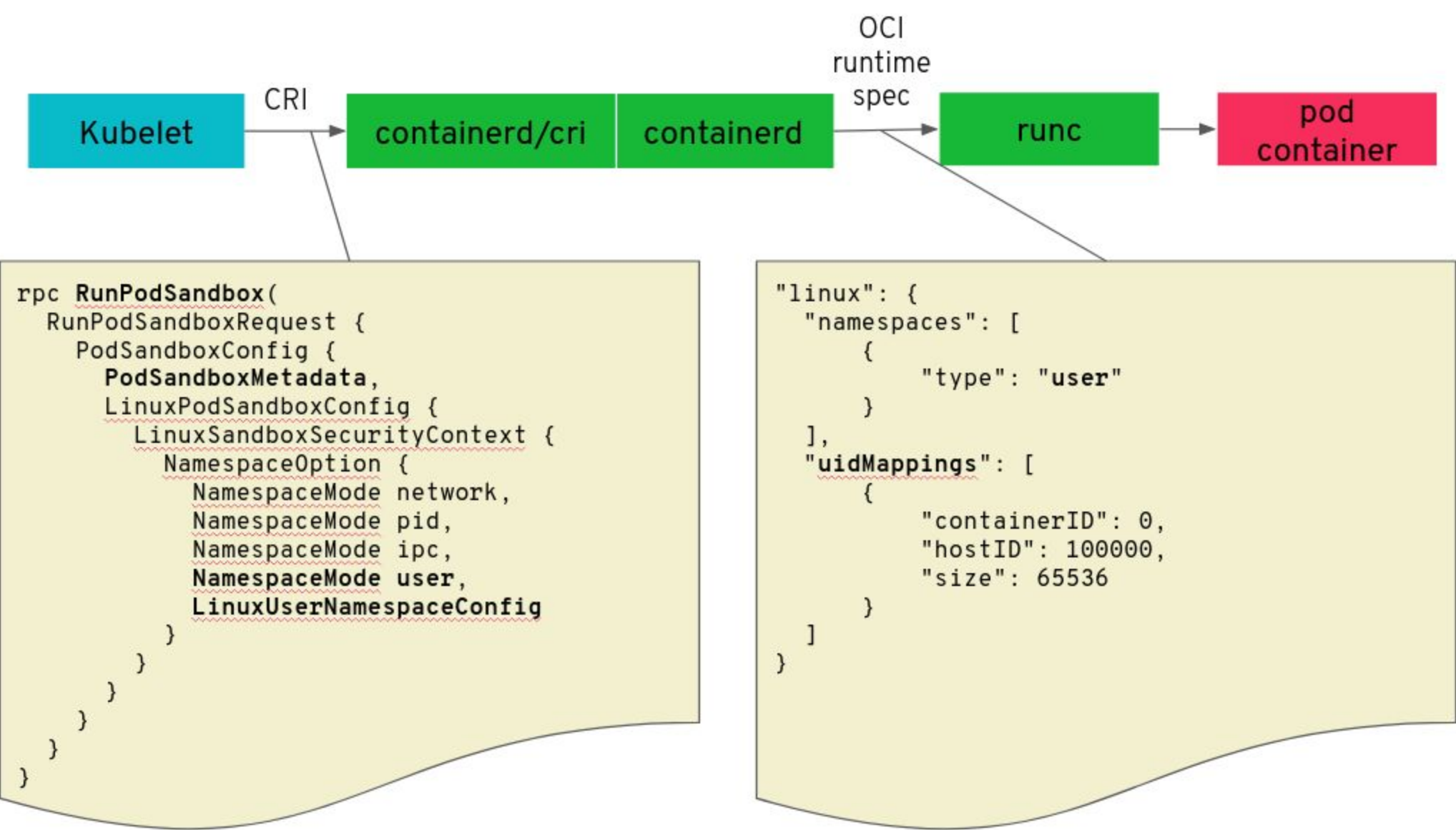
# Как устроены поды внутри узла?



# В Docker Swarm архитектура попроще



# Взаимодействие Kubernetes с containerd через CRI, конфигурация



# Нетворкинг и порты

Если в нашем приложении несколько сервисов/портов (nginx на 80, api на 8080, ...). В docker-compose мы “маппим” порты контейнера на порты хоста напрямую:

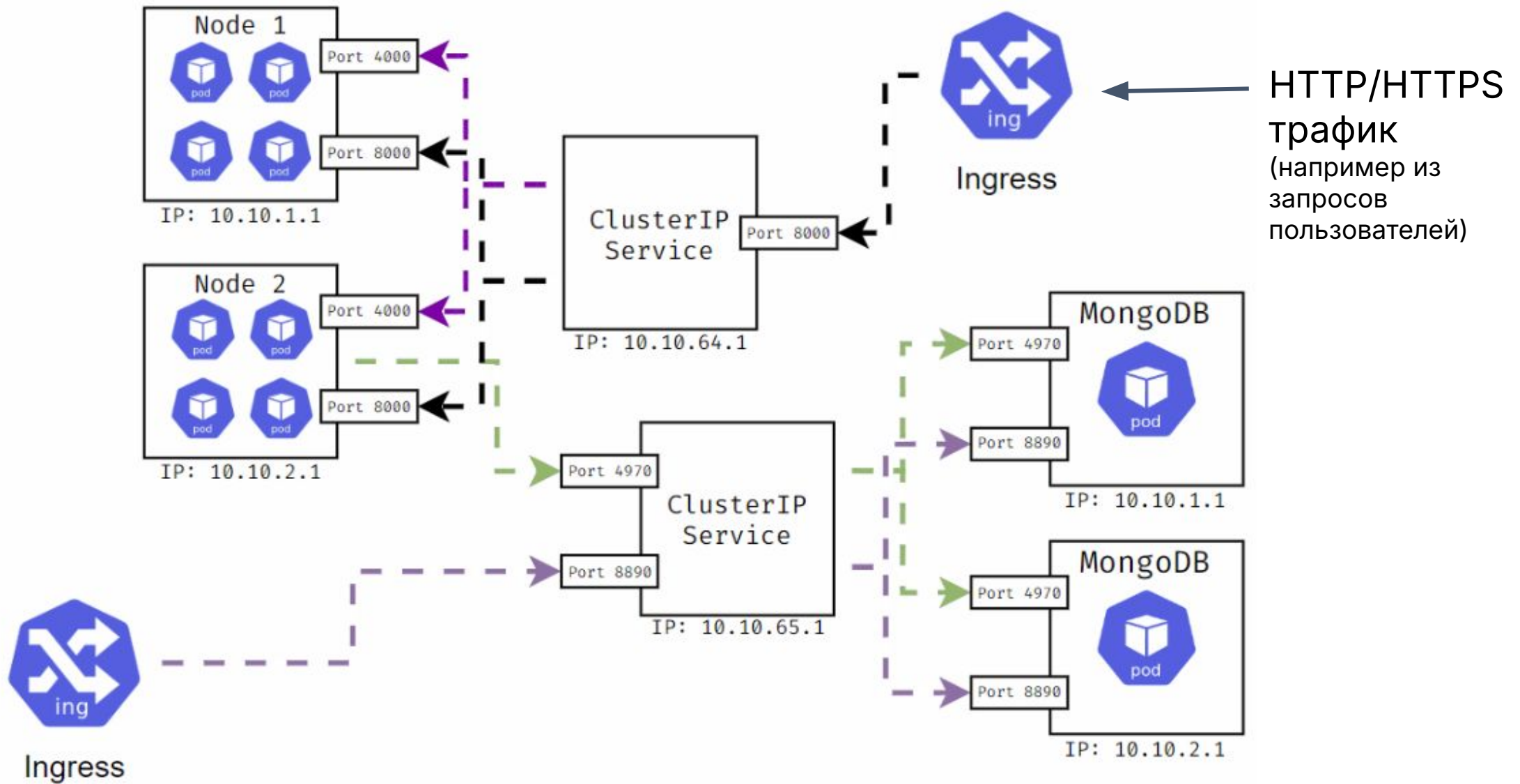
```
1 services:
2   web:
3     image: nginx
4     ports:
5       - "8080:80"
6       - "127.0.0.1:9000:9000"
7       - "3306"
```

В Kubernetes порты не надо маппировать на хост вручную — вместо этого создаётся Service (или Ingress), который абстрагирует доступ к подам, балансирует трафик и решает, как именно сделать приложение видимым снаружи.

приложение (работающее внутри подов) становится доступным:

- другим сервисам внутри кластера
- внешним пользователям / API-клиентам снаружи

# Service, Ingress



# Apptainer (бывший Singularity) — контейнеры для HPC

## Особенности:

- Разработан для HPC-кластеров
- Не требует постоянного демона (в отличие от Docker). Фоновые контейнеры запускаются без демона — просто отдельный процесс
- Контейнеры запускаются от имени обычного пользователя
- Формат образов **SIF** (Singularity Image Format) — один неизменяемый файл
- имеют полную совместимость с Docker-образами ( `apptainer pull docker:// ...` )



# Apptainer

- **Без root-привилегий.** “Fake root” через пространства имен пользователя. Процесс внутри контейнера видит uid=0, но на хосте это обычный пользователь. Это безопасно в многопользовательском окружении.
- Слои почти не используются, каждый билд - почти полностью новый образ
- Не так развит нетворкинг

# .def-файл (аналог Dockerfile)

Берём готовый Docker-образ  
с Docker Hub (или другого  
registry), Ubuntu 24.04 LTS

%post - команды во время  
сборки контейнера (= RUN)

%environment - переменные  
окружения  
(= ENV)

```
1 Bootstrap: docker
2 From: ubuntu:24.04
3 %post
4     apt-get update && apt-get install -y python3 python3-pip
5     pip3 install numpy pandas torch
6 %environment
7     export PATH=/usr/local/bin:$PATH
8 %runscript
9     python3 /app/main.py
```

# Основные команды

Скачать Docker-образ и конвертировать в .sif

```
apptainer pull tensorflow.sif docker://tensorflow/tensorflow:latest-gpu
```

Запустить (аналог docker run)

```
apptainer run tensorflow.sif          # запускает %runscript из .def файла
```

```
apptainer exec tensorflow.sif python script.py
```

```
apptainer shell tensorflow.sif        # оболочка внутри контейнера (интерактивный режим)
```

Сборка своего контейнера (из definition-файла .def)

```
sudo aptainer build myapp.sif myapp.def
```

# Podman

– альтернатива Docker, запускает контейнеры без демона

- Также как и Arptainer запускает контейнеры от имени обычного пользователя;
- Совместим с Docker-образами и командами;
- Поддерживает поды, как в Kubernetes.



podman

# Реестры контейнерных образов

*Docker-совместимые*

Docker Hub — [hub.docker.com](https://hub.docker.com)

GitHub Container Registry (GHCR) — [ghcr.io](https://ghcr.io)

Red Hat Quay / Quay.io — [quay.io](https://quay.io)

*Singularity / Apptainer (SIF)*

Sylabs Cloud Library — [cloud.sylabs.io](https://cloud.sylabs.io)

BioContainers — [biocontainers.pro](https://biocontainers.pro)

*Harbor - непубличный "self-hosted" реестр*

